Nationwide Auction System

TECHNICAL PRESENTATION

YINA CAI

Kata Overview

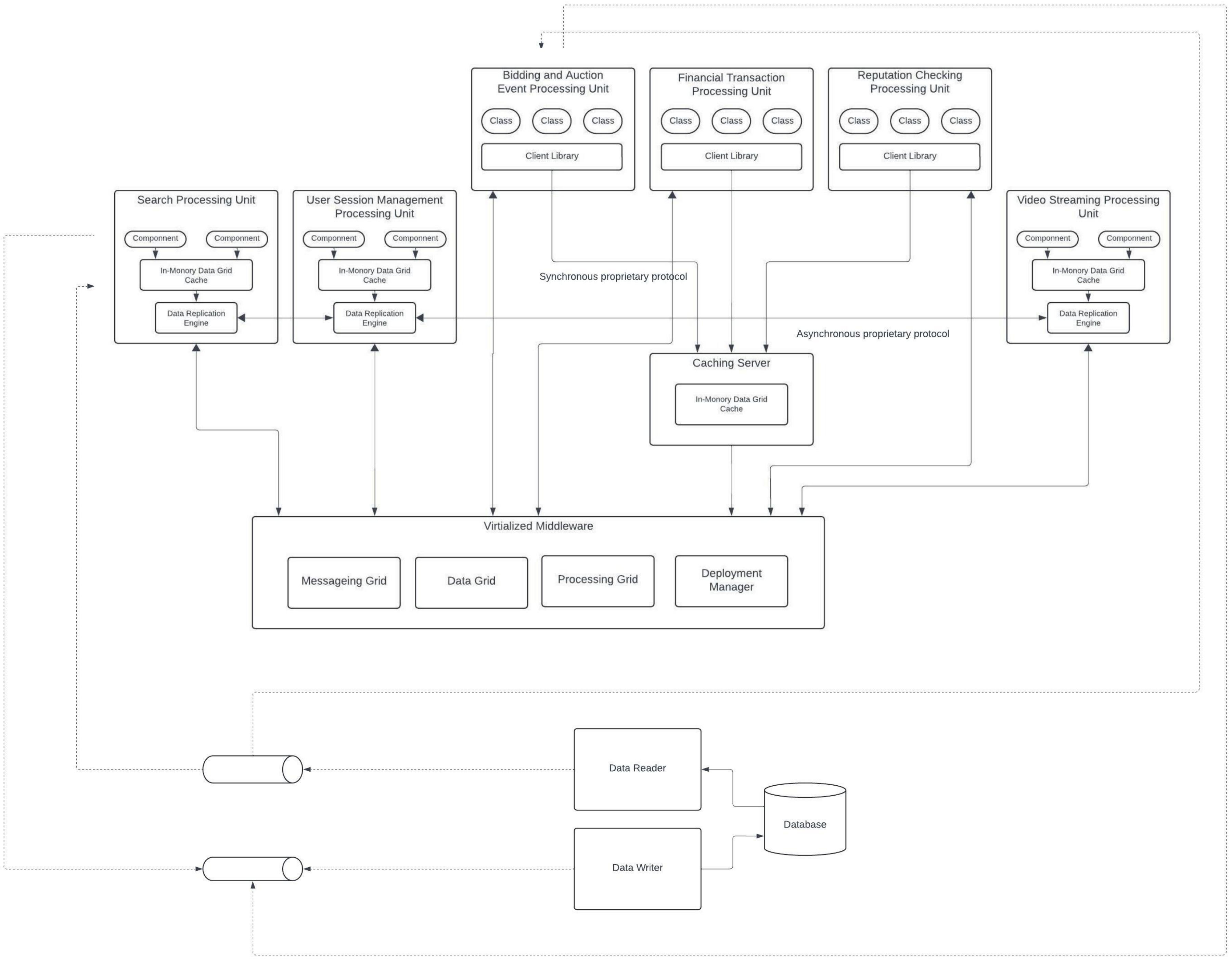
- An auction company wants to take their auctions online to a nationwide scale. This system is designed to allow users to choose an auction and participate at anywhere in real-time. This system needs to handle large volume of participants, and it offers both live and online auctions. Ideally, the system supports auctions as many simultaneous auctions as possible, and it provides post-auction video streaming. The system manage financial transitions with money exchange available, and it tracks the participate reputations. Scalability is a firm requirement because the company is expanding aggressively merging with smaller competitors and expects overseas replication.
- Characteristics: Scalability, Performance, Elasticity, Availability

Architecturally Interesting Bits

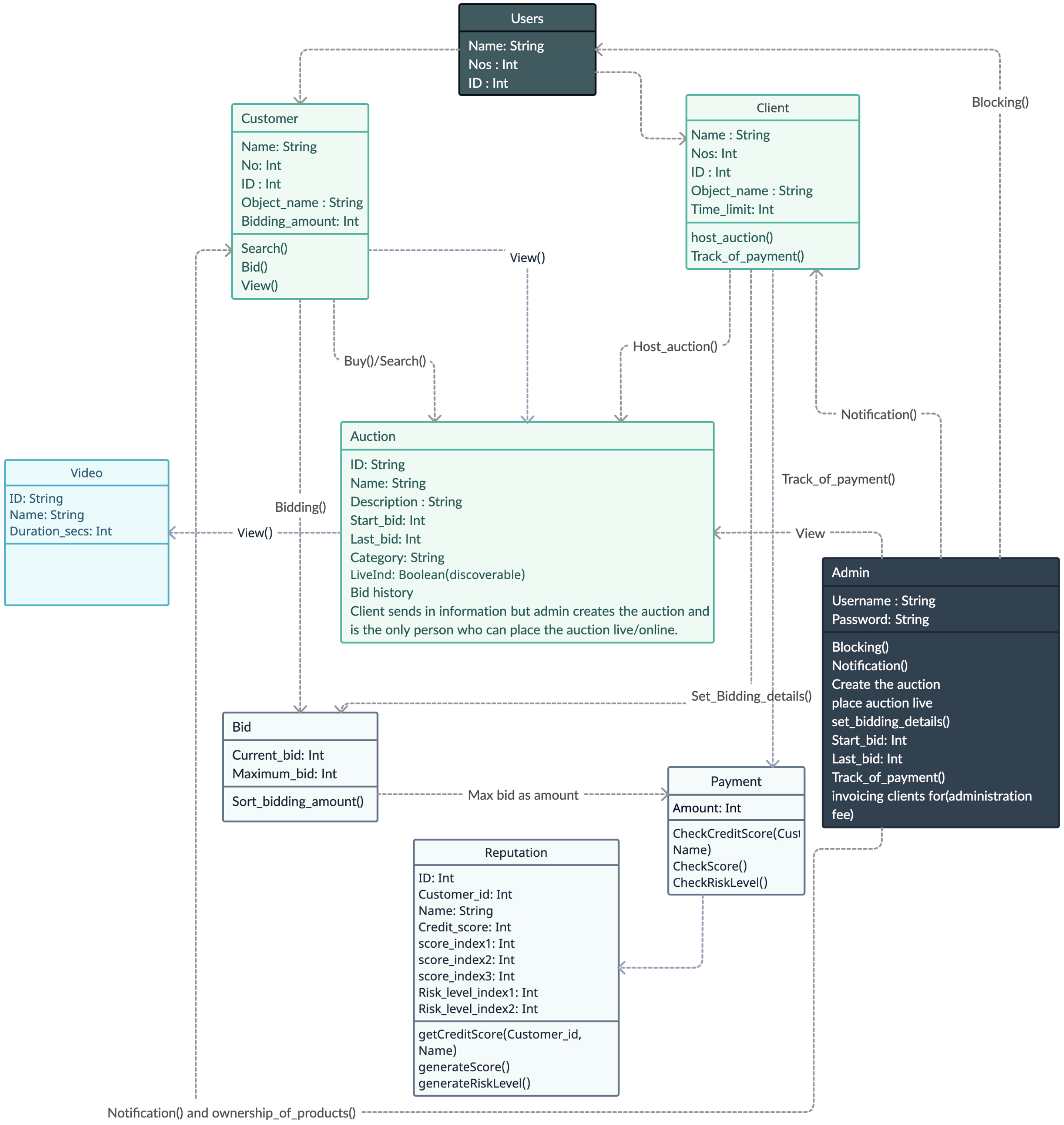
- Scalability: This system needs to handle large volume of participants and supports auctions as many simultaneous auctions as possible.
- Real-time Bidding: Both live and online auctions happen in real time, which means quick synchronous data.
- Data Consistency: Bidding, money exchange handling and reputation tracking requires data consistency.
- Extensibility: Aggressive expansion and potential replicate overseas.
- Performance and Elasticity: unpredictable spikes in user and request load.
- Reliability: With financial transactions and real time auctions, reliability is critical.

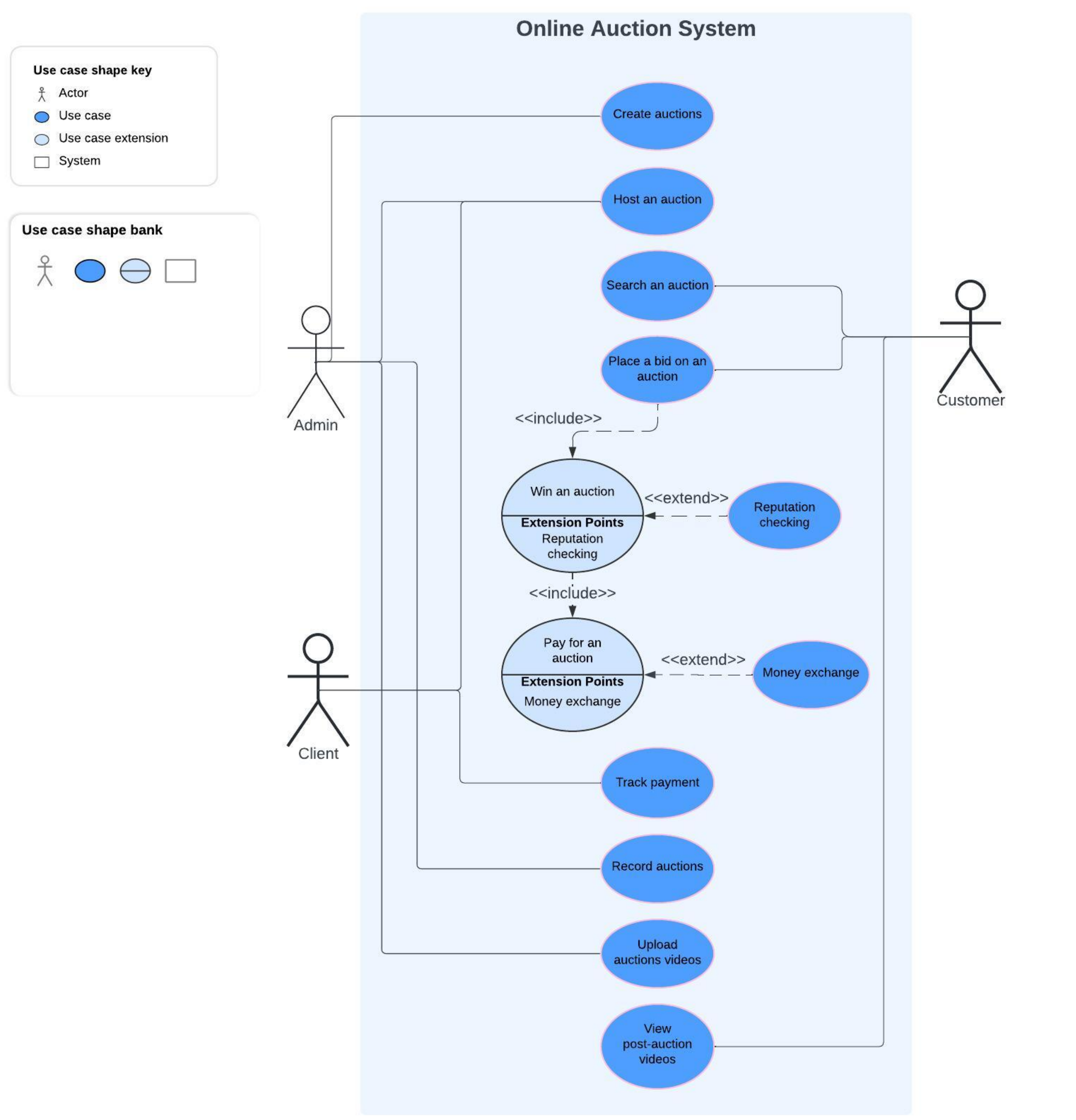
Proposed Solution with Space-based Architecture Style

- Space-based architecture is well suited for online auction system because it supports high performance, high scalability and high levels of elasticity to handle unpredictable spikes in user and request load.
- Multiple processing units can be started as the load increases; and as the auction winds down, unused processing units could be destroyed.
- Individual processing units can be devoted to each auction, ensuring consistency with bidding data.
- Due to the asynchronous nature of the data pumps, bidding data can be sent to other processing without much latency.



Bidding and Auction Event Processing Unit
Class
Class
Class
Client Library
Financial Transaction Processing Unit
Class
Class
Class
Client Library
Reputation Checking Processing Unit
Class
Class
Class
Client Library
Search Processing Unit
Componnent
Componnent
In-Monory Data Grid Cache
Data Replication Engine
User Session Management Processing Unit
Componnent
Componnent
In-Monory Data Grid Cache
Data Replication Engine
Video Streaming Processing Unit
Componnent
Componnent
In-Monory Data Grid Cache
Data Replication Engine
Synchronous proprietary protocol
Asynchronous proprietary protocol
Caching Server
In-Monory Data Grid Cache
Virtialized Middleware
Messageing Grid
Data Grid
Processing Grid
Deployment Manager
Data Reader
Data Writer
Database


Users
Name: String
Nos : Int
ID : Int
Customer
Name: String
No: Int
ID : Int
Object_name : String
Bidding_amount: Int
Search()
Bid()
View()
Client
Name : String
Nos: Int
ID : Int
Object_name : String
Time_limit: Int
host_auction()
Track_of_payment()
Blocking()
View()
Buy()/Search()
Host_auction()
Notification()
Track_of_payment()
Auction
ID: String
Name: String
Description : String
Start_bid: Int
Last_bid: Int
Category: String
LiveInd: Boolean(discoverable)
Bid history
Client sends in information but admin creates the auction and is the only person who can place the auction live/online.
Video
ID: String
Name: String
Duration_secs: Int
Bidding()
View()
View
Admin
Username : String
Password: String
Blocking()
Notification()
Create the auction
place auction live
set_bidding_details()
Start_bid: Int
Last_bid: Int
Track_of_payment()
invoicing clients for(administration fee)
Set_Bidding_details()
Bid
Current_bid: Int
Maximum_bid: Int
Sort_bidding_amount()
Max bid as amount
Payment
Amount: Int
CheckCreditScore(Cust Name)
CheckScore()
CheckRiskLevel()
Reputation
ID: Int
Customer_id: Int
Name: String
Credit_score: Int
score_index1: Int
score_index2: Int
score_index3: Int
Risk_level_index1: Int
Risk_level_index2: Int
getCreditScore(Customer_id, Name)
generateScore()
generateRiskLevel()
Notification() and ownership_of_products()



Online Auction System
Use case shape key
Actor
Use case
Use case extension
System
Use case shape bank
Create auctions
Host an auction
Search an auction
Place a bid on an auction
<<include>>
Win an auction
Extension Points
Reputation checking
<<extend>>
Reputation checking
<<include>>
Pay for an auction
Extension Points
Money exchange
<<extend>>
Money exchange
Track payment
Record auctions
Upload auctions videos
View post-auction videos
Admin
Client
Customer

Other solutions that considered but discarded

- Event-Driven Architecture: Event-driven architecture allows high scalability, performance and fault tolerance. Bidding is an event based model, it makes me to consider this architecture style at the first place. Since the auctions must be as real-time as possible, handle payment processing and requires reputation tracking, data consistency is essential. Data loss could happen between event propagation, and it’s lack of elasticity.
- Microkernel Architecture: Since the company expanding aggressively by merging with smaller competitors, which means the system needs to have an ability to combine other systems. The core kernel manages the essential auction processes, such as auction searching, bid management, and user registration, and the plugins handles payment processing, video streaming, money exchange, reputation tracking, and integrating with competitors’ system after merging. Unfortunately, microkernel architecture doesn’t not scale well. Scalability, fault tolerance, and elasticity are its weakness.

Fitness Function(s)

- Canary test: Monitoring critical metrics, e.g. unavailable, failure, response times.
- Gatling: Load testing to ensure that the system can handle thousands of participants and simultaneous auctions.
- Grafana - Latency Benchmarking: Auctions must be as real-time as possible, e.g. backend latency, connection latency.
- Cloud service availability monitor: Recommend the auction system deploy in any major cloud platform for fault tolerance support.
- Consistency checks: Run tests to verify data consistency for financial transactions and reputation checking to ensuring no race conditions or inconsistencies.

Failure Cases and Mitigation

- Node failure: If a processing unit fails, the deployment manager in virtualized middleware space ensures that another processing unit starts up to reducing down time.
  - Recommend to use cloud platforms to auto-scaling and monitor failures.
- Data collisions: Due to replication latency of the caching product.
  - Reducing the number of processing units, reducing the cache size, or a better caching product.
- Data inconsistency: If data grid fails, data inconsistency happen between replicas, and it will affect the bidding accuracy.
  - Use distributed cache instead of replicated cache for important data.

Normal state, Warning state, Alarm state

- Normal State: All nodes are functioning and no failure. User participation, bidding, and payments are processed in real-time.
- Warning State: Performance decrease with increased latency or unexpected long response times, but the system remains operational. Load balancer or additional processing units are spun up to address the load.
- Alarm State: Node failures occur(e.g. load balancer failure, backend server failure, and database failure) potentially impacts user experience or data integrity. The system triggers redundancy mechanisms to maintain operation.

Expected Data Growth and Impacts

- Number of Auctions: As the company expands overseas, the number of simultaneous auctions and auction data will grow.
- User Data: As the company grows and aggressively merging other smaller competitors, the user data grows aggressively as well. The reputation tracking will generate a large volume of data, especially as more users join.
- Media Files: Post-auction video streams will add significant storage demands.
- Impacts: With increasing users and auction, data storage will need to expand. More replication must need. This increase the infrastructure cost and data grid scalability.

Business Continuity Needs

- High availability: Using distributed cloud infrastructure ensures the auction system 99.99% uptime.
- Disaster recovery: Using distributed cloud infrastructure ensures the auction system can recover form regional failures.
- Data backup strategy: Redundancy is must have to avoid data loss when database failure happens.
- Risk management: network security for financial transitions, maintain data consistency and prevent data loss. Avoid another law suit.
- Communication: A well document architecture design and runbooks help with the system maintainability.